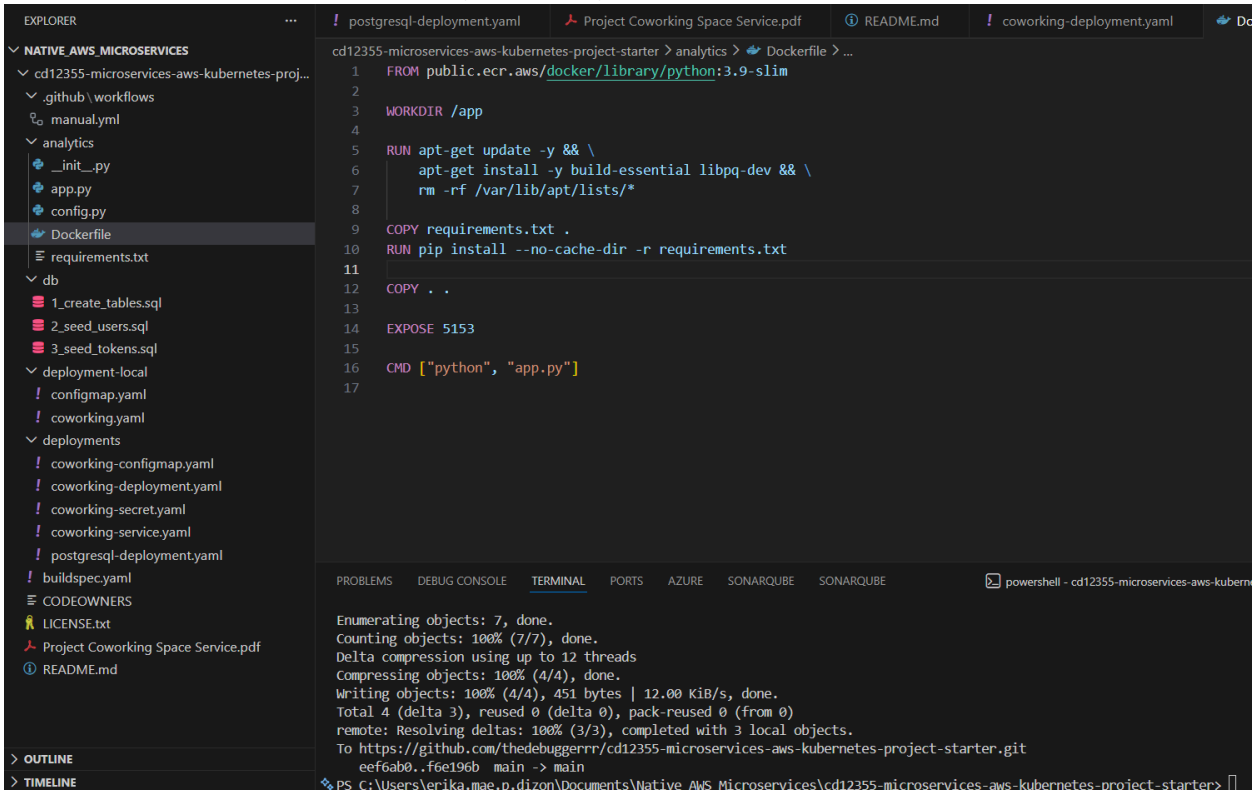
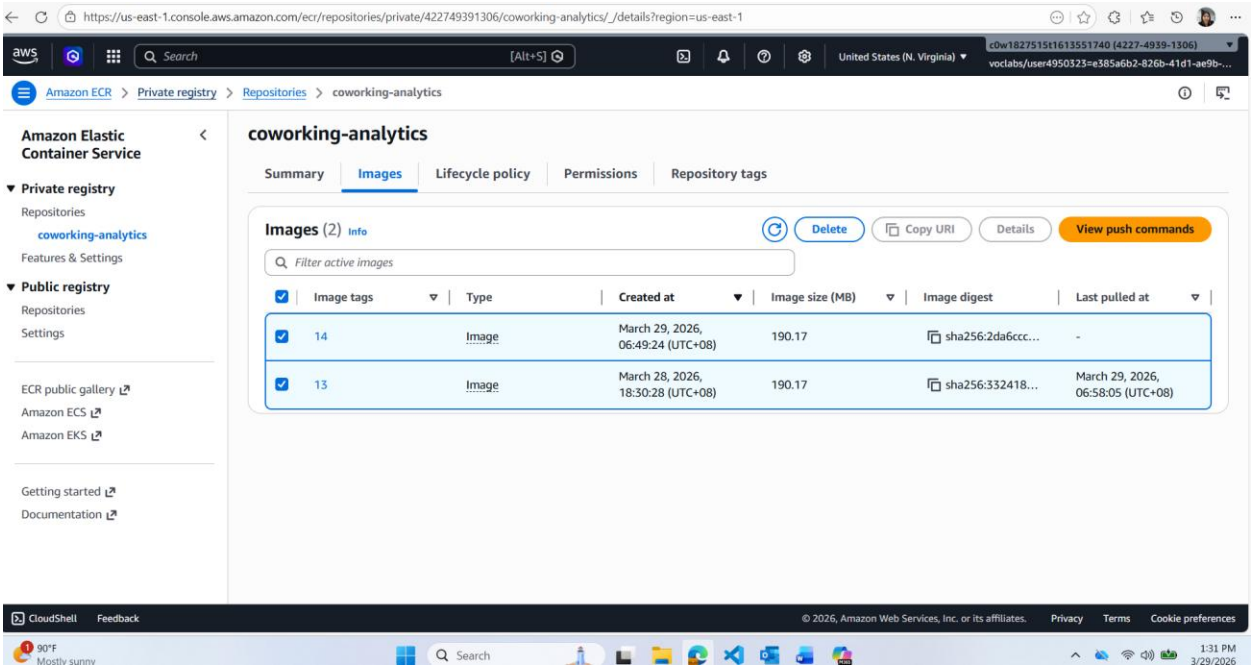
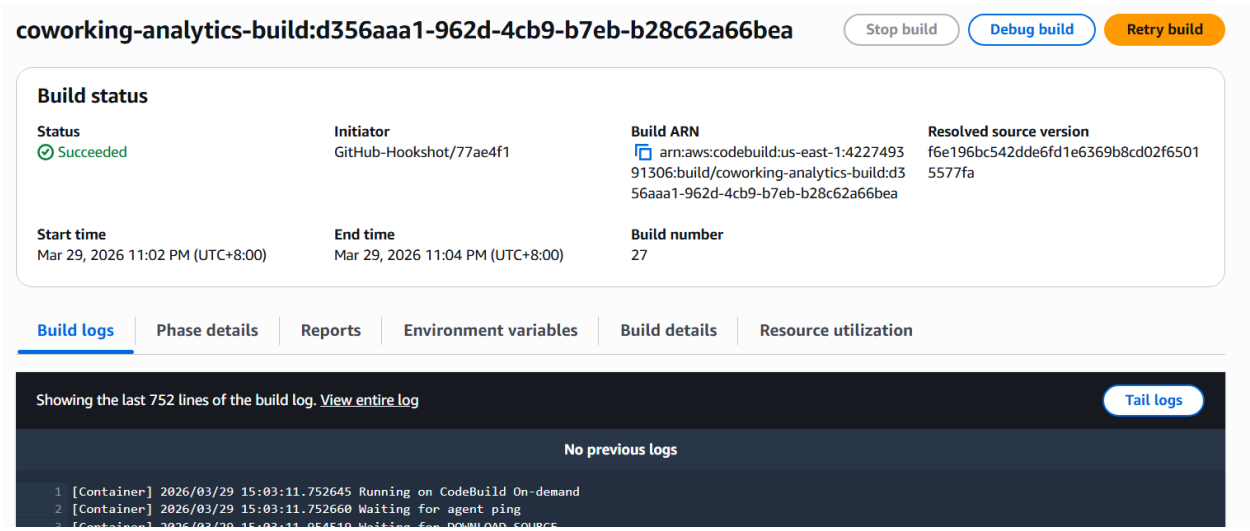
1. Docker file.
   The Dockerfile inside the analytics/ directory defines the container image for the Coworking application. It uses python:3.9-slim, installs the required dependencies, copies the application source code, exposes port 5153, and starts the service with python app.py.



2. Docker Image.
   The Amazon ECR repository coworking-analytics contains the Docker images for the application. The Images tab shows successfully pushed image tags, confirming that the application image was stored in ECR and is available for deployment.



3. Screenshot of AWS code build pipeline.
   I fixed the GitHub webhook and the build is now triggered automatically by GitHub (Initiator = GitHub-Hookshot). However, when CloudWatch logs are enabled, the build fails immediately in the QUEUED phase because the lab account explicitly denies logs:CreateLogStream for the CodeBuild service role. When CloudWatch logs are disabled, the build can complete successfully, but CodeBuild no longer provides the normal build log view. Because of this lab restriction, I am submitting the successful GitHub-triggered build details together with the ECR image evidence instead of the sample-style build log screenshot.



```
[Container] 2026/03/29 15:04:07.801264 Running command docker tag coworking-analytics:$CODEBUILD_BUILD_NUMBER $AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/coworking-analytics:$CODEBUILD_BUILD_NUMBER

[Container] 2026/03/29 15:04:07.826466 Phase complete: BUILD State: SUCCEEDED
[Container] 2026/03/29 15:04:07.826486 Phase context status code:  Message:
[Container] 2026/03/29 15:04:07.863923 Entering phase POST_BUILD
[Container] 2026/03/29 15:04:07.865173 Running command echo Build completed on `date`
Build completed on Sun Mar 29 03:04:07 PM UTC 2026

[Container] 2026/03/29 15:04:07.875129 Running command docker push $AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/coworking-analytics:$CODEBUILD_BUILD_NUMBER
```

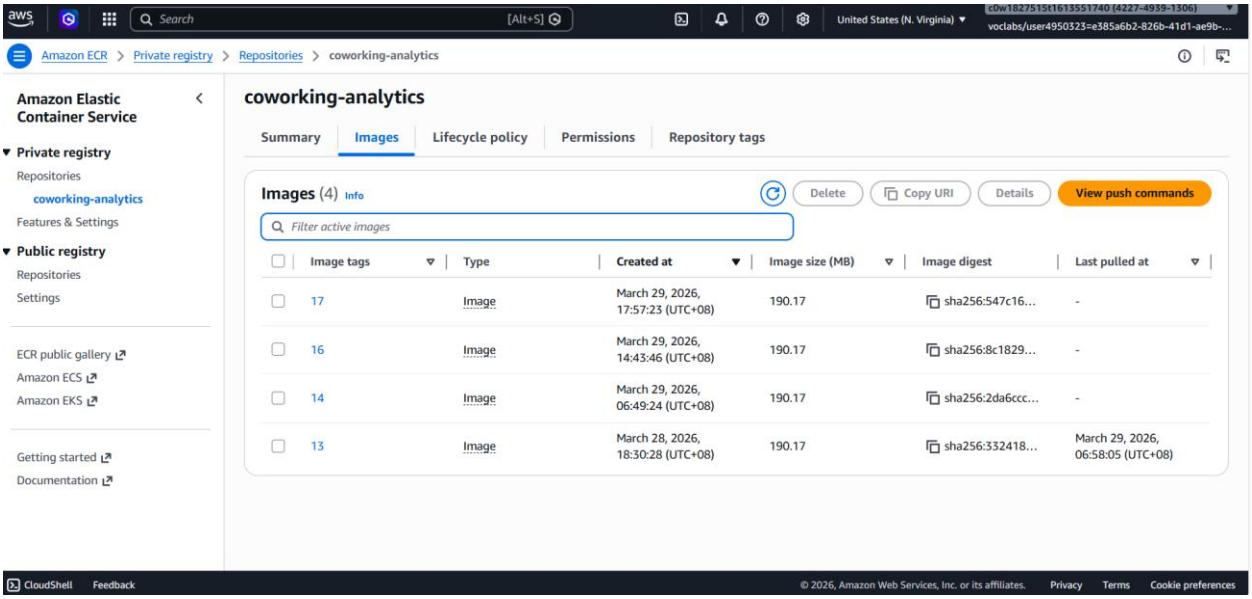
[Alt+S] United States (N. Virginia) c0w1827515t1613551740 (4227-4939-1306) voclabs/user4950323=e385a6b2-826b-41d1-ae9b-

projects > coworking-analytics-build > coworking-analytics-build:d356aaa1-962d-4cb9-b7eb-b28c62a66bea

```
de0bc4499ed1: Preparing
c8f6b54339a8: Preparing
298992e09a03: Preparing
4f237755fbae: Preparing
d7c97cb6f1fe: Preparing
c8f6b54339a8: Waiting
298992e09a03: Waiting
4f237755fbae: Waiting
d7c97cb6f1fe: Waiting
17ebedf5d2c3: Pushed
78f920aaa5fd: Pushed
c8f6b54339a8: Layer already exists
298992e09a03: Layer already exists
4f237755fbae: Layer already exists
de0bc4499ed1: Pushed
207ee931929f: Pushed
d7c97cb6f1fe: Pushed
6ce4c7f04873: Pushed
27: digest: sha256:41f6ce45ad38c9a015588637eb27e7f9912955c2d128f031e44a05f9ac228f54 size: 2206

[Container] 2026/03/29 15:04:36.991247 Phase complete: POST_BUILD State: SUCCEEDED
[Container] 2026/03/29 15:04:36.991265 Phase context status code:  Message:
[Container] 2026/03/29 15:04:37.054768 Set report auto-discover timeout to 5 seconds
[Container] 2026/03/29 15:04:37.054815 Expanding base directory path:  .
[Container] 2026/03/29 15:04:37.060508 Assembling file list
[Container] 2026/03/29 15:04:37.060526 Expanding .
[Container] 2026/03/29 15:04:37.065989 Expanding file paths for base directory .
[Container] 2026/03/29 15:04:37.066007 Assembling file list
[Container] 2026/03/29 15:04:37.066011 Expanding **/*
[Container] 2026/03/29 15:04:37.072208 No matching auto-discover report paths found
[Container] 2026/03/29 15:04:37.072230 Report auto-discover file discovery took 0.017462 seconds
[Container] 2026/03/29 15:04:37.072411 Phase complete: UPLOAD_ARTIFACTS State: SUCCEEDED
[Container] 2026/03/29 15:04:37.072420 Phase context status code:  Message:
```

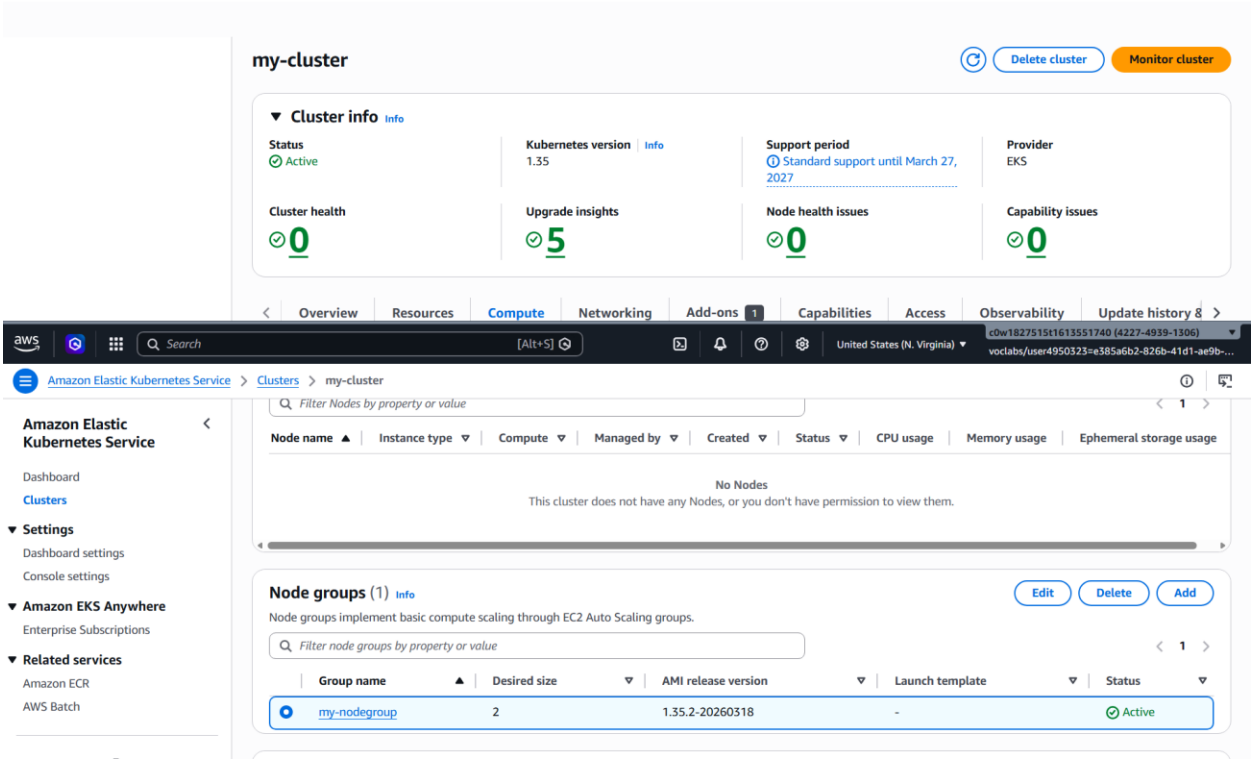
4. Screenshot of AWS ECR repository for application repository.
   The Amazon ECR repository coworking-analytics shows multiple successfully pushed Docker images under the Images tab, confirming that the application container image was stored in ECR.



5. Screenshot of kubectl get svc.
   The kubectl get svc output confirms that the Kubernetes services were created successfully, including the Coworking application service and the PostgreSQL database service. The output shows the external LoadBalancer for the application and the internal ClusterIP services for both the application and database.

```
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices> kubectl get svc
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP                                                                  PORT(S)          AGE
coworking           LoadBalancer   10.100.175.220   a050776aa629044cba18f40d4b0582fc-1637300564.us-east-1.elb.amazonaws.com   5153:30906/TCP   3h5m
coworking-service   ClusterIP      10.100.116.153   <none>                                                                       5153/TCP         3h52m
kubernetes          ClusterIP      10.100.0.1       <none>                                                                       443/TCP          14h
postgresql-service  ClusterIP      10.100.119.186   <none>                                                                       5432/TCP         14h
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices>
```

6. Kubect get nodes.
   The EKS worker nodes were checked in **Amazon EKS** → **my-cluster** → **Compute**, and the related EC2 instances were verified to be running. Since the cluster nodes were not available, the EC2 worker instances were rebooted. After rebooting, kubectl get nodes confirmed that both worker nodes returned in the **Ready** state.



After the checking the node- group in the cluster , and try to run this.

```
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter> kubectl get nodes 2>$null
NAME                                       STATUS   ROLES    AGE   VERSION
ip-192-168-23-99.ec2.internal              Ready    <none>   15m   v1.35.2-eks-f69f56f
ip-192-168-41-66.ec2.internal              Ready    <none>   15m   v1.35.2-eks-f69f56f
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter>
```

7. kubectl get pods.
   Sometimes the AWS lab session expires, which can cause terminal commands such as kubectl or aws to stop working correctly. Before running commands again, make sure the AWS lab session is active in the AWS Console and verify that the required resources are still running. In Amazon EKS, open **my-cluster** → **Compute** and confirm that the node group and EC2 worker nodes are in a healthy running state. If needed, refresh the AWS session credentials first, then rerun the terminal commands.

```
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
coworking-7f4c46dcc8-pjqx2    1/1     Running   0          127m
postgresql-66cb897f7f-s6kts   1/1     Running   0          127m
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter>
```

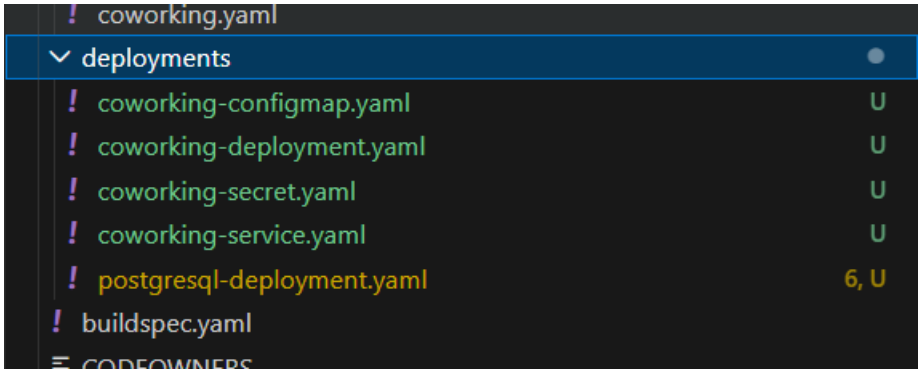
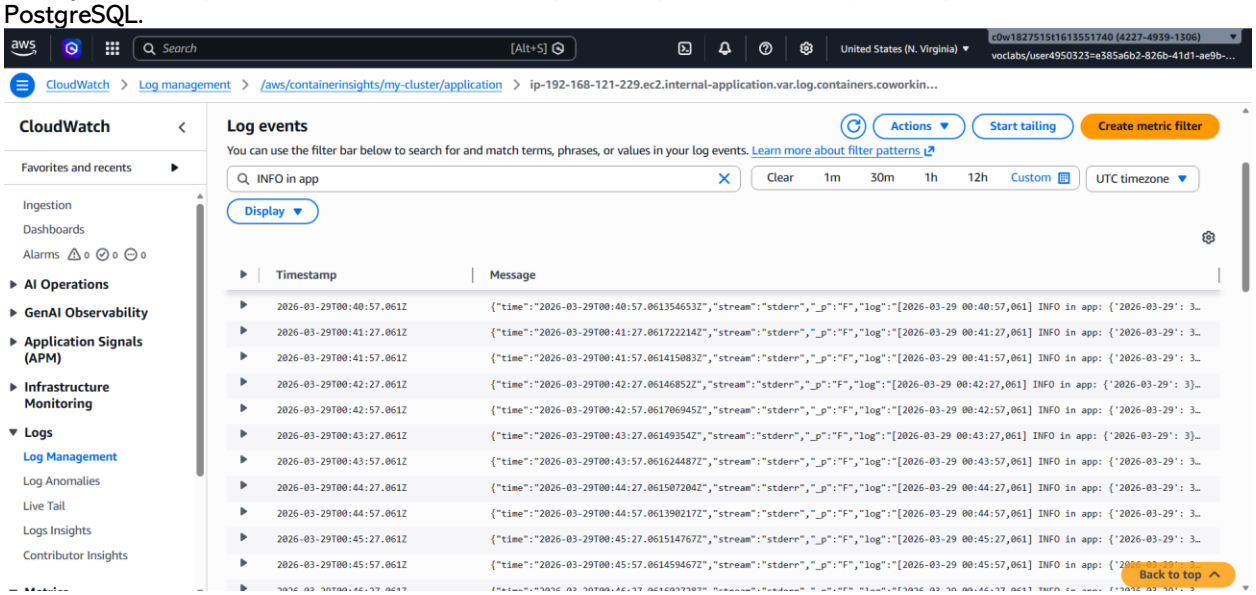
8. kubectl describe svc postgresql -service.
   The output confirms that the Kubernetes database service was created successfully in the **default** namespace with service type **ClusterIP**. The service is configured to expose port **5432/TCP**, which is the default PostgreSQL database port. The output also shows an active **endpoint** (192.168.38.155:5432), confirming that the service is correctly connected to the running PostgreSQL pod inside the cluster.

```
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter> kubectl describe svc postgresql-service 2
>$null
Name:              postgresql-service
Namespace:         default
Labels:            <none>
Annotations:       <none>
Selector:          app=postgresql
Type:              ClusterIP
IP Family Policy:  SingleStack
IP Families:       IPv4
IP:                10.100.119.186
IPs:               10.100.119.186
Port:              <unset>  5432/TCP
TargetPort:        5432/TCP
Endpoints:         192.168.38.155:5432
Session Affinity:  None
Events:            <none>
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter>
```

9. kubectl describe deployment coworking.
   The kubectl describe deployment coworking output confirms that the Coworking application deployment was successfully created in Kubernetes. The deployment is available, the replica is running, and the application container is configured correctly.

```
Events:            <none>
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter> kubectl describe deployment coworking
Name:                   coworking
Namespace:              default
CreationTimestamp:      Sat, 28 Mar 2026 19:37:05 +0800
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 6
Selector:               service=coworking
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:       service=coworking
  Annotations:  kubectl.kubernetes.io/restartedAt: 2026-03-29T08:37:25+08:00
  Containers:
   coworking:
    Image:      422749391306.dkr.ecr.us-east-1.amazonaws.com/coworking-analytics:13
    Port:       5153/TCP
    Host Port:  0/TCP
    Environment Variables from:
      coworking-config  ConfigMap  Optional: false
    Environment:
      DB_PASSWORD:  <set to the key 'DB_PASSWORD' in secret 'coworking-secret'>  Optional: false
    Mounts:         <none>
  Volumes:          <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets:  <none>
NewReplicaSet:   coworking-6dc5cbc896 (1/1 replicas created)
Events:          <none>
PS C:\Users\erika.mae.p.dizon\Documents\Native_AWS_Microservices\cd12355-microservices-aws-kubernetes-project-starter>
```

10. All Kubernetes config files used for deployment (ie YAML files) stored in a directory called deployments/.
    All Kubernetes YAML files used for deployment were placed inside the deployments/ directory, including the Coworking application deployment, service, ConfigMap, Secret, and PostgreSQL deployment configuration.



11. CLoudwatch.
    CloudWatch Container Insights logs were checked in /aws/containerinsights/my-cluster/application. The coworking log stream showed repeated log entries every 30 seconds, including INFO in app: {'2026-03-29': 3}, confirming that the application is running normally and successfully reading data from PostgreSQL.